



Compare Gram-Schmidt and Householder Orthogonalization Algorithms

Cleve Corner: Cleve Moler on Mathematics and Computing

Classic Gram-Schmidt, Modified Gram-Schmidt, and Householder reflections can be used for the same task. The three algorithms have very different rou

Contents

- G. W. (Pete) Stewart
- Classic Gram-Schmidt
- $X = QR$
- Modified Gram-Schmidt
- Householder Reflections
- Householder QR factorization
- Comparison
- Reference

G. W. (Pete) Stewart

My colleague and friend G. W. Stewart is a Distinguished University Professor Emeritus at the De University of Maryland. Everybody knows him as “Pete”. He has never been able to satisfactorily e me. It somehow goes back through his father to his grandfather and maybe great grandfather, w

Pete has written several books on numerical linear algebra. For my blog today I am going to rely c pseudocode from his book “Matrix Algorithms, Volume I: Basic Decompositions”. His pseudocod

Classic Gram-Schmidt

The classic Gram-Schmidt algorithm is the first thing you might think of for producing an orthogo vector in your data set, remove its projection onto the data set, normalize what is left, and add it t code. X is the original set of vectors, Q is the resulting set of orthogonal vectors, and R is the set c upper triangular matrix.

```
type gs
function [Q,R] = gs(X)
% Classical Gram-Schmidt. [Q,R] = gs(X);
% G. W. Stewart, "Matrix Algorithms, Volume 1", SIAM, 1998.
[n,p] = size(X);
Q = zeros(n,p);
R = zeros(p,p);
for k = 1:p
    Q(:,k) = X(:,k);
    if k ~= 1
        R(1:k-1,k) = Q(:,k-1)'*Q(:,k);
        Q(:,k) = Q(:,k) - Q(:,1:k-1)*R(1:k-1,k);
    end
    R(k,k) = norm(Q(:,k));
    Q(:,k) = Q(:,k)/R(k,k);
end
end
```

$X = QR$

The entire process can be expressed by matrix multiplication, with orthogonal Q and upper trian

$$X = QR$$

Modified Gram-Schmidt

This is a rather different algorithm, not just a simple modification of classical Gram-Schmidt. The emerging set of vectors instead of against the original set. There are two variants, a column-wise one. They produce the same results, in different order. Here is the column version,

```
type mgs
function [Q,R] = mgs(X)
% Modified Gram-Schmidt. [Q,R] = mgs(X);
% G. W. Stewart, "Matrix Algorithms, Volume 1", SIAM, 1998.
[n,p] = size(X);
Q = zeros(n,p);
R = zeros(p,p);
for k = 1:p
    Q(:,k) = X(:,k);
    for i = 1:k-1
        R(i,k) = Q(:,i)'*Q(:,k);
        Q(:,k) = Q(:,k) - R(i,k)*Q(:,i);
    end
    R(k,k) = norm(Q(:,k))';
    Q(:,k) = Q(:,k)/R(k,k);
end
end
```

Householder Reflections

A Householder reflection H transforms a given vector x into a multiple of a unit vector e_k while $Hx = \pm\sigma e_k$ where $\sigma = \|x\|$.

The matrix H is never formed. The reflection is expressed as a rank-one modification of the identity

$$H = I - uu^T \text{ where } \|u\| = \sqrt{2}$$

(The use of $\sqrt{2}$ here is Pete's signature normalization.)

```
type housegen
function [u,nu] = housegen(x)
% [u,nu] = housegen(x)
% Generate Householder reflection.
% G. W. Stewart, "Matrix Algorithms, Volume 1", SIAM, 1998.
% [u,nu] = housegen(x).
% H = I - uu' with Hx = -+ nu e_1
% returns nu = norm(x).
u = x;
nu = norm(x);
if nu == 0
    u(1) = sqrt(2);
    return
end
u = x/nu;
if u(1) >= 0
    u(1) = u(1) + 1;
    nu = -nu;
else
    u(1) = u(1) - 1;
end
u = u/sqrt(abs(u(1)));
end
```

Now to apply H to any other y , compute the inner product

$$\tau = u^T y$$

and then simply subtract

$$Hy = x - \tau u$$

Householder QR factorization

This program does not actually compute the QR orthogonalization, but rather computes R and a vector u to generate the Householder reflectors whose product is Q.

```
type hqrd

function [U,R] = hqrd(X)
    % Householder triangularization. [U,R] = hqrd(X);
    % Generators of Householder reflections stored in U.
    % H_k = I - U(:,k)*U(:,k)'.
    % prod(H_m ... H_1)X = [R; 0]
    % where m = min(size(X))
    % G. W. Stewart, "Matrix Algorithms, Volume 1", SIAM, 1998.
    [n,p] = size(X);
    U = zeros(size(X));
    m = min(n,p);
    R = zeros(m,m);
    for k = 1:min(n,p)
        [U(k:n,k),R(k,k)] = housegen(X(k:n,k));
        v = U(k:n,k)'*X(k:n,k+1:p);
        X(k:n,k+1:p) = X(k:n,k+1:p) - U(k:n,k)*v;
        R(k,k+1:p) = X(k,k+1:p);
    end
end
```

Comparison

All three of these algorithms compute Q and R that do a good job of reproducing the data X, that is

- $Q * R$ is always close to X for all three algorithms.

On the other hand, their behavior is very different when it comes to producing orthogonality. Is Q'

- Classic Gram-Schmidt. Usually very poor orthogonality.
- Modified Gram-Schmidt. Depends upon condition of X. Fails completely when X is singular.
- Householder triangularization. Always good orthogonality.

```
type compare.m
```

```

function compare(X);
% compare(X)
% Compare three QR decompositions,
%
I = eye(size(X));

%% Classic Gram Schmidt
[Q,R] = gs(X);
qrerr_gs = norm(Q*R-X,inf)/norm(X,inf);
ortherr_gs = norm(Q'*Q-I,inf);

%% Modified Gram Schmidt
[Q,R] = mgs(X);
qrerr_mgs = norm(Q*R-X,inf)/norm(X,inf);
ortherr_mgs = norm(Q'*Q-I,inf);

%% Householder QR Decomposition
[U,R] = hqrd(X);
QR = R;
E = I;
for k = size(X,2):-1:1
    uk = U(:,k);
    QR = QR - uk*(uk'*QR);
    E = E - uk*(uk'*E) - (E*uk)*uk' + uk*(uk'*E*uk)*uk';
end
qrerr_h = norm(QR-X,inf)/norm(X,inf);
ortherr_h = norm(E-I,inf);

%% Report results
fprintf('QR error\n')
fprintf('Classic:      %10.3e\n',qrerr_gs)
fprintf('Modified:     %10.3e\n',qrerr_mgs)
fprintf('Householder:  %10.3e\n',qrerr_h)
fprintf('\n')
fprintf('Orthogonality error\n')
fprintf('Classic:      %10.3e\n',ortherr_gs)
fprintf('Modified:     %10.3e\n',ortherr_mgs)
fprintf('Householder:  %10.3e\n',ortherr_h)

```

Well conditioned.

```
compare(magic(7))
```

QR error

Classic: 1.726e-16

Modified: 6.090e-17

Householder: 3.654e-16

Orthogonality error

Classic: 3.201e+00

Modified: 1.534e-15

Householder: 1.069e-15

Poorly conditioned, nonsingular.

```
compare(hilb(7))
```

QR error
Classic: 5.352e-17
Modified: 5.352e-17
Householder: 7.172e-16

Orthogonality error
Classic: 5.215e+00
Modified: 1.219e-08
Householder: 1.686e-15

Singular.

```
compare(magic(8))
```

QR error
Classic: 1.435e-16
Modified: 8.540e-17
Householder: 2.460e-16

Orthogonality error
Classic: 5.413e+00
Modified: 2.162e+00
Householder: 2.356e-15

Reference

G. W. Stewart, "Matrix Algorithms, Volume 1: Basic Decompositions", SIAM, 1998.

<https://www.amazon.com/Matrix-Algorithms-1-Basic-Decompositions/dp/0898714141>

Published with MATLAB® R2016a

Get the MATLAB code
[mathworks.com](https://www.mathworks.com)

© 1994-2024 The MathWorks, Inc. MATLAB and Simulink are registered trademarks of The MathWorks, Inc. See [mathworks.com/trademarks](https://www.mathworks.com/trademarks) for a list of additional trademarks. Other product or brand names may be trademarks or registered trademarks of their respective holders.